

Desenvolvimento de uma API com integração *frontend* em *react*® para registro automatizado de presença em disciplinas utilizando ESP32 e leitor RFID

Francisco Gleidson N. de Queiroz¹, Fábio Timbó Brito²

¹Eixo da Computação - Instituto Federal do Ceará (IFCE)
Maracanaú - CE - Brazil

²Eixo da Indústria - Instituto Federal do Ceará (IFCE)
Maracanaú - CE - Brazil

francisco.gleidson.nascimento00@aluno.ifce.edu.br, fabio@ifce.edu.br

Abstract. *This article presents a development of an API integrated with a React® frontend, designed for the automation of attendance for a school subject, utilizing the ESP32 microcontroller and a radio frequency identification (RFID) reader, model RFID-6300. This API, in addition to facilitating the registration of students and courses, allows for querying data related to student absences. Additionally, the system includes login authentication features, ensuring secure access to the platform. The purpose of this work is providing a comprehensive strategy for attendance registration in educational institutions, integrating hardware, API, and an interactive user interface. We detail in the article the system architecture, the stages of implementing the API and the React® architecture by user interface, as well as showcasing practical usage examples. The results demonstrate the efficacy and feasibility of this innovative solution to automate the process of attendance registration in educational settings, standing out as a significant contribution to efficient academic management.*

Resumo. *Este artigo apresenta o desenvolvimento de uma API integrada a um frontend em React®, destinada à automação do registro de presença em disciplinas escolares, utilizando o microcontrolador ESP32 e um leitor de identificação por radiofrequência (RFID), modelo RFID-6300. Esta API, além de possibilitar o cadastro de alunos e disciplinas, permite a consulta de dados referentes a ausências estudantis. Adicionalmente, o sistema incorpora recursos de autenticação de login, assegurando o acesso seguro à plataforma. O propósito deste trabalho é fornecer uma estratégia abrangente para o registro de presença em instituições de ensino, integrando hardware, API e uma interface de usuário interativa. Detalhamos no artigo a arquitetura do sistema, as etapas de implementação da API e da interface de usuário em arquitetura React®. Os resultados alcançados evidenciam a eficácia e a viabilidade dessa solução para automatizar o processo de registro de presença em contextos educacionais, destacando-se como um contributo significativo para a gestão acadêmica eficiente.*

1. Introdução

A automação de processos crescente no campo da educação pode otimizar a gestão e melhorar a eficiência de várias operações. Um desses processos, que pode significativamente se beneficiar dessa tendência, é o registro de presença de alunos em

sala de aula, um procedimento que, tradicionalmente, é conduzido manualmente, exigindo tempo e estando sujeito a falhas.

Neste artigo, descrevemos o desenvolvimento de uma *API*(*Application Programming Interface*) integrada a uma interface de usuário em *React*®, criada para automatizar o registro de presença em disciplinas, utilizando o microcontrolador *ESP32* já que possui recursos integrados como wifi que é necessário para o projeto e um leitor *RFID*(*Radio Frequency Identification*) modelo *RFID-6300*. Essa solução proposta compreende uma interface interativa que facilita o cadastro de alunos e disciplinas, consulta de faltas e fornece recursos de autenticação de login para assegurar o acesso seguro aos dados.

Desse modo, pretendemos oferecer uma abordagem abrangente ao desenvolvimento da *API* com integração *frontend* em *React*® para o registro automatizado de presença em disciplinas, utilizando o *ESP32* e o leitor *RFID*.

2. Referencial Teórico

O controle e registro de frequência em instituições educacionais têm sido objeto de vários estudos que buscam utilizar a tecnologia para aumentar a eficiência e a precisão desses processos. De Moura Pereira et al. (2012) propuseram um sistema para acompanhar a frequência escolar de forma automatizada, utilizando cartões *RFID*, com a operacionalidade do sistema garantida pelo Java EE.

A automação do registro de frequência utilizando a biometria tem sido um foco em muitos desses estudos. Fontes and de Almeida Júnior (2021) apresentaram um sistema para automação de frequência em sala de aula usando reconhecimento biométrico. Da mesma forma, Silva et al. (2017a) propuseram um protótipo de um sistema de identificação biométrica utilizando a plataforma *Arduino* para monitorar a frequência discente. O uso de biometria pode melhorar a precisão e a eficiência do registro de frequência; no entanto, os desafios de privacidade e segurança associados à biometria devem ser cuidadosamente considerados [Fontes e de Almeida Júnior 2021].

Da Silva et al. (2018) apresentaram o "BioRegSchool", um sistema de registro de frequência escolar que também utiliza a leitura biométrica. Este sistema compartilha os mesmos benefícios e desafios que outros sistemas de biometria [da Silva et al., 2018].

Prestes et al. (2022) focaram em analisar e desenvolver um sistema computacional em Java para controle da frequência dos alunos da Universidade São Judas Tadeu utilizando cartões *RFID* como meio de identificação. A linguagem de programação Java® é amplamente utilizada e bem documentada, o que pode facilitar a implementação do sistema.

Por último, Silva et al. (2017b) descreveram o desenvolvimento de um sistema de controle eletrônico de frequência e acesso não focando na forma que se daria o registro da frequência, apenas informando que seria um meio eletrônico como biometria, cartão *RFID*, etc. Assim como o trabalho de De Moura Pereira et al. (2012), porém se difere por focar no desenvolvimento de uma aplicação web.

Em conclusão, o controle e registro de frequência em instituições educacionais têm sido cada vez mais automatizados com o uso da tecnologia. A biometria, em particular, tem sido usada para melhorar a eficiência e a precisão do registro de frequência. No entanto, os desafios de privacidade, segurança e higiene associados à biometria devem ser cuidadosamente gerenciados.

Uma comparação das aplicações apresentadas acima com relação ao método de autenticação, infraestrutura e o tipo de aplicação se encontra na Tabela 1.

Tabela 1. Comparação entre os projetos apresentados

Referência	Registro de Frequência	Processamento	Aplicação
[Oliveira, 2011]	Biométrico	Servidor local	Frequência Alunos
[de Moura Pereira et al., 2012]	Cartão RFID	Java EE	Frequência Escolar
[Fontes and de Almeida Júnior, 2021]	Biométrico	Arduino	Frequência em Sala de aula
[Silva et al., 2017a]	Biométrico	Arduino	Monitoramento discentes
[da Silva et al., 2018]	Biométrico	NodeMCU	Registro escolar
[Prestes et al., 2022]	Cartão RFID	Arduino Uno R3, Java	Universidade
[Silva et al., 2017b]	Eletrônico	WEB	Controle de frequência e acesso às portas.

Apesar das soluções apresentadas para o controle de frequência demonstrarem avanços importantes, elas possuem limitações significativas. Dentre estas, destacam-se: as preocupações com privacidade e o custo elevado associado à autenticação biométrica, as exigências de infraestrutura específica e a manutenção associada. Portanto, surge a necessidade de uma solução mais versátil, acessível, de fácil implementação e que respeite a privacidade dos usuários.

Assim será montado um circuito utilizando o leitor *RFID* juntamente com o *ESP32* e um *Display LCD* que se comunicará com uma *API* desenvolvida em *Node.js* e terá uma página desenvolvida com *REACT®* e utilizando javascript. Assim o *ESP32* enviará a presença utilizando uma rota da *API* a qual ainda não exige autenticação para as solicitações e na página que também se comunicará com a *API* será possível cadastrar login para professores, cadastrar alunos e disciplinas e consultar os alunos faltosos para serem lançados no sistema frequência da universidade.

3. Materiais e Métodos

Para o desenvolvimento deste projeto, foram utilizados os seguintes materiais, ferramentas e componentes:

Backend em *Node.js* (Node.js, 2023) responsável por criar, editar, consultar e excluir os dados na planilha do *google* que por padrão, é aplicado um limite de taxa de 10.000.000 unidades de cota a cada 100 segundos por projeto de produtor de serviços, uma unidade de cota é consumida a cada chamada para [services.check](#) e a cada operação informada como [services.report](#). A linguagem de programação utilizada foi o *JavaScript* também foi utilizado o *framework Express.js* e as bibliotecas *bcrypt*, *express*, *Google Spreadsheet*, *moment*, *cors*. As rotas utilizadas no *backend* se encontram na Tabela 2.

Tabela 2. Rotas utilizadas no *backend*

Método	Rota da API	Funcionalidade
<i>POST</i>	/post	Insere um aluno na planilha
<i>POST</i>	/chamada	Realiza a inclusão da presença na planilha
<i>GET</i>	/get	Consulta todos os alunos cadastrados na planilha
<i>PUT</i>	/put	Altera um aluno da planilha
<i>DELETE</i>	/delete	delete um aluno da planilha
<i>GET</i>	/disciplinas	Consultas as disciplinas cadastradas
<i>GET</i>	/status	Verifica o estado da API
<i>POST</i>	/criarlogin	Adicionar um <i>login</i> na planilha
<i>POST</i>	/validalogin	Validar um <i>login</i> para acesso a página.
<i>POST</i>	/cadastrardisciplina	Cadastrar disciplinas na planilha
<i>POST</i>	/consultafaltas	Consulta os alunos faltosos.

Frontend utilizando o *React*® (React®, 2023) é responsável por fazer a intermediação entre o usuário e a API, sendo usadas as bibliotecas *styled*, *axios* e *DatePicker* que tem por objetivos:

1. A biblioteca *Styled* é uma biblioteca de estilização em *JavaScript* para aplicações *web*. Seu principal objetivo é permitir que os desenvolvedores apliquem estilos de forma mais intuitiva e dinâmica aos componentes de suas aplicações.
2. *Axios* é uma biblioteca de *HTTP* baseada em *Promises*, que permite a realização de requisições assíncronas em aplicações *web*. Seu objetivo principal é simplificar o processo de comunicação entre o cliente e o servidor, facilitando a obtenção e o envio de dados através de requisições *HTTP*.
3. A biblioteca *DatePicker* é uma biblioteca que facilita a seleção de datas em aplicações *web*. Seu objetivo é fornecer um componente de calendário interativo que permite aos usuários escolher uma data específica de forma intuitiva.

Para a interface do usuário foram criados os seguintes componentes as suas

respectivas imagens de tela podem ser vistas abaixo:

O componente “*SearchComponent*” é responsável pela interface de consulta dos alunos faltosos onde é feita uma solicitação a *API* que compara os alunos cadastrados na disciplina com os alunos que registraram presença na data e disciplina escolhidas retornando assim os alunos faltosos, podemos ver a tela do componente na Figura 1.

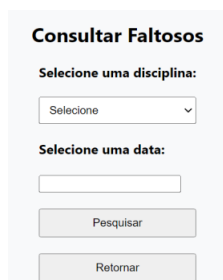


Figura 1. Tela do componente *SearchComponent*

O componente “*LoginComponent*” é responsável pela interface de login onde irá realizar uma solicitação a *API* que informará se o login informado terá acesso ao sistema e suas funcionalidades, podemos ver a tela do componente na Figura 2.

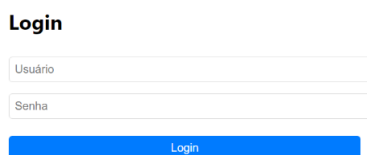


Figura 2. Tela do componente *LoginComponent*

O componente “*HomePageComponent*” é a interface onde temos todas as funcionalidades do sistema como criar login, cadastrar aluno, cadastrar disciplina, consultar presença e desconectar, podemos ver a tela do componente na Figura 3.

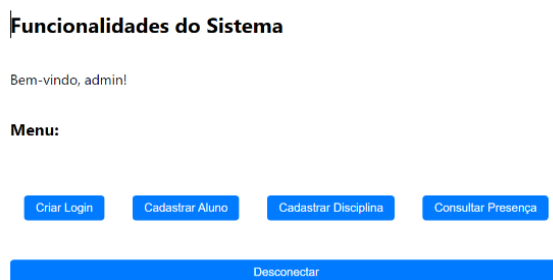


Figura 3. Tela do componente *HomePageComponent*

O componente “*CriarLoginComponent*” é responsável pela interface de criação de logins solicitando a *API* a criação dos logins que terão acesso ao sistema e suas funcionalidades informando os campos login, senha e nome. Podemos ver a tela do componente na Figura 4.

Criar Login

Login

Senha

Nome

Criar Login

Voltar

Figura 4. Tela do componente *CriarLoginComponent*

O componente “*CadastrarAlunoComponent*” é responsável pela interface de cadastrar aluno onde será solicitado a *API* que seja feito cadastro dos alunos na planilha informando os campos cartão, matrícula, nome e disciplina. Podemos ver a tela do componente na Figura 5.

Cadastrar Aluno

Cartão:

Matrícula:

Nome:

Selecione uma disciplina:

Selecione

Cadastrar Aluno

Voltar

Figura 5. Tela do componente *CadastrarAlunoComponent*

O componente “*CadastrarDisciplinaComponent*” é responsável pela interface de cadastrar as disciplinas ofertadas onde será solicitado a *API* que seja feito o cadastro da disciplina na planilha informando os campos nome da disciplina, os dias da semana em que ocorre o turno onde temos “M” para manhã, “T” para tarde ou “N” para noite e o horário sendo “AB” ou “CD” referente a cada turno. Podemos ver a tela do componente na Figura 6.

Cadastrar Disciplina

Disciplina:

Dias da Semana:

☐ Segunda-feira ☐ Terça-feira ☐ Quarta-feira ☐ Quinta-feira ☐ Sexta-feira ☐ Sábado

Horário:

Selecione

Turno:

Selecione

Cadastrar Disciplina Retornar

Figura 6. Tela do componente *CadastrarDisciplinaComponent*

O componente “*ChamadaComponent*” é responsável pela interface de registrar a presença caso o aluno tenha esquecido o seu cartão *RFID* solicitando a *API* que registre a presença do aluno informando apenas os campos matrícula do aluno que é selecionada de uma relação dos alunos cadastrados, pois a data, hora e dia da semana já é incluída automaticamente. Podemos ver a tela do componente na Figura 7.

Figura 7. Tela do componente ChamadaComponent

No circuito para registro de presença foram utilizados os componentes microcontrolador *ESP32*, *display LCD 16x2 com I2C*, leitor *RFID-6300* e um regulador de tensão para 5V e uma bateria de 9V.

O esquema do circuito e o mesmo montado pode ser visto na figura 8:

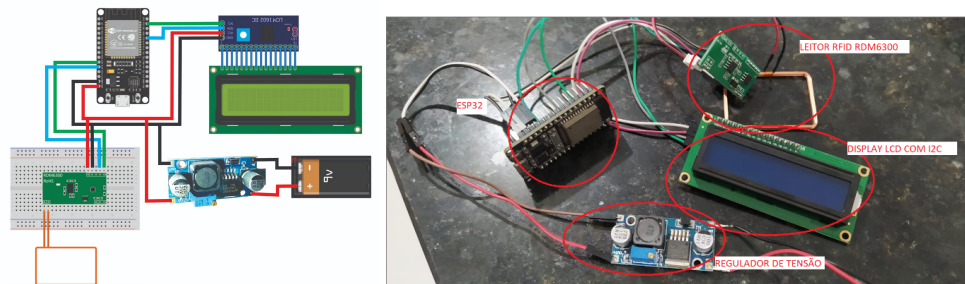


Figura 8. Esquema do circuito de registro de presença e o mesmo montado

A Figura 9 descreve basicamente o fluxograma de funcionamento do projeto:

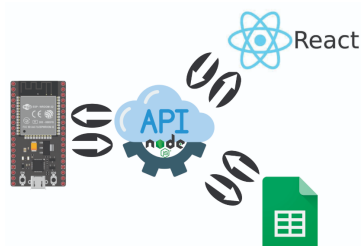


Figura 9. Fluxograma do Projeto

Neste projeto, o *backend* foi desenvolvido utilizando o *Node.js* com o *framework Express.js*. Foram criadas rotas para realizar operações de criação, atualização, exclusão e consulta de usuários, disciplinas, faltas e logins. Além disso, foram implementadas rotas para validação de login e obtenção do status da *API*.

No *frontend*, foi utilizado o *React®* para criar os componentes de interface do usuário. Foram desenvolvidas telas de busca, *login*, página inicial, criação de *login*, cadastro de aluno e cadastro de disciplina. As bibliotecas *styled*, *axios* e *DatePicker* foram utilizadas para estilização, comunicação com o *backend* e seleção de datas, respectivamente.

Para o registro de presença, foi montado um circuito utilizando o *ESP32*, o display *LCD 16x2* com *I2C* e o leitor *RFID* *RFID-6300*. O regulador de tensão foi utilizado para fornecer energia de 5V para os componentes.

Esses materiais, ferramentas e componentes foram utilizados em conjunto para desenvolver a solução proposta, permitindo o registro automatizado de presença realizado pelo *ESP32* que realizará a leitura do cartão *RFID* através do leitor de cartão e envia os dados por rota específica da *API* para armazenar os dados na planilha, o cadastro de alunos e disciplinas, e a consulta de faltas serão solicitadas pelo *frontend* em *React*® a *API* também através de rotas específicas para cada solicitação que fará o armazenamento, edição ou consulta a planilha. A integração entre o *backend* em *Node.js*, o *frontend* em *React*® e o circuito com o *ESP32*, leitor *RFID* e display *LCD* possibilita a interação eficiente do sistema.

4. Resultados

Para avaliar a eficácia e funcionalidade da solução proposta, o *frontend* do aplicativo foi hospedado gratuitamente na plataforma *Vercel* (*Vercel*, 2023) e o *backend* do aplicativo foi implantado gratuitamente na infraestrutura de servidores *Fly.io* (*Fly.io*, 2023), foram realizados testes e experimentos no *LINC*(laboratório de instrumentação e controle) do eixo da indústria do *IFCE*(Instituto Federal do Ceará) Maracanaú. Os resultados obtidos foram os seguintes:

A *API* desenvolvida em *Node.js* mostrou-se eficiente no processamento das requisições do *frontend*. As rotas implementadas permitiram a criação, atualização, exclusão e consulta de usuários, disciplinas, faltas e logins. Além disso, as funcionalidades de validação de login e obtenção do status da *API* foram bem-sucedidas conforme planilhas da Figura 10:

	A	B	C	D		A	B	C	D	E
1	disciplina	diasemana	horario	turno	1	cartao	data	hora	diasemana	disciplina
2	TESTE AB M	seg-ter-qua-qui-sex	AB	M	2	21635203	26/06/2023	10:35:48	seg	TESTE CD M
3	TESTE CD M	seg-ter-qua-qui-sex	CD	M	3	21635203	27/06/2023	08:57:42	ter	TESTE AB M
4	TESTE AB T	seg-ter-qua-qui-sex	AB	T	4	21635203	27/06/2023	08:58:32	ter	TESTE AB M
5	TESTE CD T	seg-ter-qua-qui-sex	CD	T	5	21635203	27/06/2023	10:53:58	ter	TESTE CD M
6	dispo2	seg-qua	AB	M	6	21635203	27/06/2023	10:55:52	ter	TESTE CD M

	A	B	C	D		A	B	C
1	cartao	matricula	nome	disciplina	1	login	senha	nome
2	324235	201510	Gleidson	TESTE AB M	2	admin	\$2b\$10\$llpq0KIAIRq3w3FojOKRu7lI3BLiI5m2nXCVejae2RvYsBKbFdBm	Administrador
3	324235	201510	Gleidson	TESTE CD M	3	teste	\$2b\$10\$AaXN.D6A11bqH8DnRxUTrvuQDXR0qsTNxsSokLqfhGETCMgqKl	teste
4	324235	201510	Gleidson	TESTE AB T				
5	324235	201510	Gleidson	TESTE CD T				
6	324235	201510	fulna	TESTE CD M				

alunos	presencas	disciplinas	login
--------	-----------	-------------	-------

Figura 10. Planilhas de dados

O circuito montado com o *ESP32*, leitor *RFID* e display *LCD* funcionou conforme o esperado. O leitor *RFID* realizou a leitura dos cartões de identificação dos alunos de forma precisa, permitindo o registro automatizado de presença. O display *LCD* exibiu corretamente as informações de confirmação ou registro de presença. Podemos ver o protótipo finalizado na Figura 11.

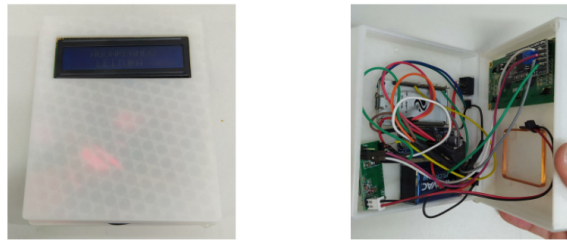


Figura 11. Protótipo do circuito

Funcionalidade da interface de usuário em *React*[®]: Os componentes desenvolvidos em *React*[®] foram capazes de interagir de forma eficiente com a *API*. A tela de busca permitiu a consulta de alunos e disciplinas, enquanto a tela de *login* possibilitou o acesso seguro aos dados. As telas de criação de *login*, cadastro de aluno e cadastro de disciplina também funcionaram corretamente.

Em geral, os resultados demonstraram que a solução proposta foi capaz de automatizar o registro de presença em disciplinas, facilitar o cadastro de alunos e disciplinas, e fornecer consultas de faltas de forma eficiente. A integração entre o *backend* em *Node.js*, o *frontend* em *React*[®] e o circuito com o *ESP32*, leitor *RFID* e *display LCD* permitiu uma interação fluida e segura do sistema.

Esses resultados são promissores e indicam que a solução desenvolvida pode trazer benefícios na forma de realizar a lista de presença em sala de aula. Este projeto demonstrou a viabilidade e o potencial da automação do registro de presença em disciplinas por meio da integração de tecnologias como o *ESP32*, leitor *RFID* e o uso de uma *API* em *Node.js* com um *frontend* em *React*[®]. Futuros trabalhos podem explorar essas melhorias e aprimoramentos, além de avaliar a adoção e aceitação dessa solução para integração com os sistemas do campus.

5. Conclusão

A automação do registro de presença em disciplinas por meio da integração de tecnologias como o *ESP32*, leitor *RFID* e uma *API* desenvolvida em *Node.js*, aliada ao *frontend* em *React*[®], demonstrou ser uma solução para otimizar o processo da realização da lista de presença em sala de aula. Os resultados obtidos durante os testes e experimentos revelaram a eficácia da solução, destacando sua capacidade de agilizar a gestão e melhorar a precisão das operações relacionadas ao registro de presença.

A utilização do *ESP32* em conjunto com o leitor *RFID* e o *display LCD* permitiu a leitura precisa dos cartões de identificação dos alunos e o registro automatizado de presença. A interação entre o *backend* em *Node.js* e o *frontend* em *React*[®] proporcionou uma experiência de usuário intuitiva e eficiente, com recursos como busca de alunos e disciplinas, cadastro de novos logins, e consulta de faltas.

No entanto, é importante ressaltar que existem possíveis melhorias a serem consideradas, como a implementação de recursos adicionais de segurança, a otimização do desempenho da *API* e a melhoria da interface do usuário. A criptografia de senhas e a proteção dos dados dos alunos é fundamental para garantir a confidencialidade e integridade das informações. Além disso, a otimização do desempenho da *API* e a melhoria contínua da interface de usuário como também a integração desta solução com

o sistemas do campus como a *intranet*, *Qacadêmico* ou *SUAP* podem ser consideradas em trabalhos futuros para aprimorar ainda mais a solução.

Em conclusão, o desenvolvimento da API com integração *frontend* em *React*® para o registro automatizado de presença em disciplinas, utilizando o *ESP32*, leitor RFID e as tecnologias mencionadas, representa um avanço no campo da automação de processos educacionais. Essa solução oferece benefícios significativos, como a melhoria da eficiência e a garantia de um registro preciso de presença. Com possíveis melhorias e aprofundamentos futuros, acredita-se que essa solução possa ser adotada com sucesso em diversas salas de aula, contribuindo para uma experiência mais moderna, eficiente e segura no controle de presença dos alunos.

Referências

- da Silva, I. A., Valim, P. R. O., and Bortoluzzi, F. (2018). Bioregschool–sistema de registro de frequência escolar com leitura biométrica. *Anais do Computer on the Beach*, pages 771–780.
- de Moura Pereira, A. P., de Oliveira, J. A., Carvalho, M. M.M. B. B., Ataíde, W. A., and Lima, E. F. d. A. (2012). Proposta de um sistema para acompanhamento automatizado da frequência escolar. In VII CONNEPI-Congresso Norte Nordeste de Pesquisa e Inovação .
- Fly.io. (2023). Infraestrutura de servidores para implantação de aplicações. Recuperado de <https://fly.io/>
- Fontes, V. P. and de Almeida Júnior, P. L. (2021). Sistema para automação de frequência em sala de aula com reconhecimento biométrico. *Revista Semiárido De Visu*, 9(3):243–256.
- Node.js. (2023). Documentação. Recuperado de <https://nodejs.org/documentation/>
- Oliveira, I. C. (2011). Sistema web de registro de presença via leitura biométrica de impressão digital para instituições de ensino.
- Prestes, J. D. d. O., Walder, G. J., and Teixeira, I. d. S. (2022). Análise e desenvolvimento de um sistema computacional em Java para controle da frequência dos alunos da universidade são Judas Tadeu.
- React®. (2023). *React*® - Uma biblioteca JavaScript para construir interfaces de usuário. Recuperado de <https://reactjs.org/>
- Silva, E., Chaves, A., and Batista, C. (2017a). Protótipo de um sistema de identificação biométrica utilizando a plataforma arduino para monitoramento eficiente da frequência discente. VI Encontro Anual de Tecnologia da Informação do Oeste Goiano (ENATI), pages 52–55.
- Silva, E. et al. (2017b). Desenvolvimento de um sistema de controle eletrônico de frequência e acesso.
- Vercel. (2023). Plataforma de hospedagem para aplicações *frontend*. Recuperado de <https://vercel.com/>